

Fire Control System

Modélisation fonctionnelle, réseau de Petri et vérification formelle

Rapport final du projet de programmation fonctionnelle

Étudiants : Paul Pitiot
Maxime Crayssac
Arthur Neuez

Technologie : Scala, Akka Typed, Kafka, réseaux de Petri

Dépôt : <https://github.com/mcrayssac/Fire-Control-System>

Résumé

Ce rapport présente la conception et la validation d'un système de contrôle de tir modélisé comme un système critique concurrent. Le projet combine une implémentation distribuée en Scala avec Akka Typed, une représentation formelle par réseau de Petri et une vérification exhaustive des propriétés de sûreté et de vivacité. L'objectif n'est pas uniquement de simuler un cycle de tir nominal, mais de démontrer que les états dangereux sont inatteignables dans le modèle retenu : tir sans verrouillage, tir sans munition, tir sans autorisation, conflit entre tir et rechargement, stock négatif ou blocage global.

La démarche suit six axes : état de l'art, modélisation fonctionnelle, traduction vers un modèle formel, vérification des propriétés, comparaison entre modèle et implémentation, puis mode d'exécution interactif. La partie centrale du rapport est consacrée à la vérification formelle. Elle détaille l'exploration de l'espace d'états, les invariants structurels, les invariants métier, les propriétés LTL, la bornitude, la vivacité et l'absence de deadlock. Les résultats obtenus sont cohérents avec les objectifs de sûreté du projet : 111 marquages atteignables sont explorés, aucun deadlock n'est détecté, dix invariants métier et six propriétés temporelles sont satisfaits.

Table des matières

Résumé	i
1 Introduction	1
1.1 Contexte	1
1.2 Objectifs et méthode	1
2 État de l’art	2
2.1 Vérification formelle	2
2.2 Réseaux de Petri	2
2.3 Logique temporelle linéaire	3
2.4 Modèle d’acteurs	3
3 Architecture fonctionnelle du FCS	4
3.1 Choix d’architecture	4
3.2 Acteurs	4
3.3 Cycle nominal	5
3.4 Gestion des erreurs et journalisation	5
4 Modèle formel par réseau de Petri	6
4.1 Définition du modèle	6
4.2 Correspondance avec Akka	8
4.3 Hypothèses d’abstraction	8
5 Vérification formelle	9
5.1 Objectif de la vérification	9
5.2 Pipeline de vérification	9
5.3 Exploration de l’espace d’états	9
5.4 Invariants structurels	10
5.5 Invariants métier	11
5.6 Propriétés LTL	11
5.7 Bornitude	12
5.8 Vivacité et T-invariant	13
5.9 Synthèse de vérification	13
6 Simulation comparée Petri Net / Akka	14
6.1 Méthodologie	14
6.2 Résultats	14
6.3 Commandes d’exécution	15
7 Mode live interactif	16
7.1 Objectif	16
7.2 Fonctionnement	16

8 Conclusion	17
A Couverture des objectifs	18

Table des figures

4.1	Réseau de Petri du Fire Control System	7
5.1	Chaîne de vérification du FCS	9

Liste des tableaux

2.1	Opérateurs LTL utilisés	3
3.1	Responsabilités des acteurs Akka	4
3.2	Situations d’erreur prises en charge	5
4.1	Places du réseau de Petri	6
4.2	Transitions du réseau de Petri	7
4.3	Correspondance entre acteurs et réseau de Petri	8
5.1	Résultats de l’exploration exhaustive	10
5.2	Invariants métier vérifiés	11
5.3	Propriétés LTL vérifiées	12
5.4	Bornes maximales des places	12
5.5	Synthèse des résultats formels	13
6.1	Extrait du mapping transition / événement Akka	14
6.2	Résultats des scénarios de conformité	15
6.3	Commandes principales	15
7.1	Fonctionnalités du mode live	16
A.1	Correspondance avec les objectifs du projet	18

Chapitre 1

Introduction

1.1 Contexte

Un système de contrôle de tir (*Fire Control System*, FCS) appartient à la famille des systèmes critiques : une action incorrecte peut entraîner des conséquences irréversibles. La sûreté du logiciel ne peut donc pas être ramenée à quelques scénarios de test. Le système doit empêcher par construction les situations dangereuses, notamment un tir sans cible verrouillée, sans munition disponible ou sans autorisation.

Le projet étudié adopte une double approche. D'une part, le comportement opérationnel est réalisé avec le modèle d'acteurs Akka Typed, qui permet de représenter naturellement la concurrence entre capteur, suivi balistique, gestion des munitions, commande et supervision. D'autre part, les comportements critiques sont abstraits dans un réseau de Petri fini afin de permettre une exploration exhaustive de l'espace d'états.

1.2 Objectifs et méthode

Les objectifs du projet sont couverts par six livrables techniques : un état de l'art, un modèle Akka fonctionnel, une traduction vers un réseau de Petri, une vérification de propriétés, une simulation comparée entre réel et formel, et un mode interactif. Le fil conducteur du rapport est la traçabilité entre ces livrables : chaque composant implémenté est relié à une place, une transition ou une propriété du modèle formel.

La méthode de validation est la suivante :

1. concevoir une architecture fonctionnelle concurrente ;
2. identifier les états et transitions critiques ;
3. construire un réseau de Petri fini ;
4. explorer l'ensemble des marquages atteignables ;
5. vérifier des invariants de sûreté et des propriétés LTL ;
6. comparer les traces produites par le modèle formel et l'implémentation Akka.

Chapitre 2

État de l'art

2.1 Vérification formelle

La vérification formelle consiste à prouver mathématiquement qu'un système respecte certaines propriétés. Contrairement au test logiciel, qui observe un nombre fini de scénarios, elle raisonne sur l'ensemble des comportements représentés par un modèle. Cette approche est particulièrement adaptée aux domaines critiques comme l'avionique, le nucléaire ou les systèmes militaires. L'échec du vol 501 d'Ariane 5, causé par un dépassement d'entier non détecté, illustre la nécessité d'une analyse systématique des comportements logiciels [6].

Deux familles d'approches dominent. Le *model checking* explore automatiquement un modèle fini et fournit des contre-exemples en cas de violation. Le *theorem proving* permet de raisonner sur des espaces plus généraux, mais demande une expertise logique plus importante et davantage d'intervention humaine [3, 2]. Le FCS étudié ici se prête au *model checking* : son réseau de Petri produit 111 marquages atteignables, ce qui rend possible une exploration exhaustive.

2.2 Réseaux de Petri

Avant d'introduire l'écriture mathématique, il faut lire un réseau de Petri comme une carte des situations possibles du système. Les *places* sont les états ou ressources observables, par exemple être au repos, avoir une cible verrouillée ou posséder des munitions. Les *transitions* sont les actions autorisées qui font évoluer le système. Les *jetons* indiquent ce qui est vrai à un instant donné : un jeton dans une place de contrôle signifie que cette phase est active ; plusieurs jetons dans une place ressource représentent une quantité disponible.

Un réseau de Petri est alors résumé par le quintuplet :

$$N = (P, T, Pre, Post, M_0)$$

où P est l'ensemble des places, T l'ensemble des transitions, Pre et $Post$ décrivent les arcs entrants et sortants, et M_0 est le marquage initial [7, 8]. Les jetons portés par les places représentent l'état courant. Une transition t est franchissable dans un marquage M si toutes ses préconditions sont satisfaites :

$$\forall p \in P, \quad M(p) \geq Pre(t, p)$$

Cette condition signifie simplement qu'une action ne peut être exécutée que si chaque place nécessaire contient assez de jetons. Dans notre projet, cela revient par exemple à dire qu'un tir ne peut pas passer la synchronisation si l'autorisation ou la munition chargée manque.

Son franchissement produit alors le nouveau marquage :

$$M'(p) = M(p) - Pre(t, p) + Post(t, p)$$

On retire les jetons consommés par l'action, puis on ajoute ceux produits après l'action. Cette règle est la version formelle d'un changement d'état : quitter l'état *Ready_To_Fire*, entrer dans l'état *Firing*, et ajouter un événement dans la file Kafka.

La matrice d'incidence $C = Post - Pre$ résume l'effet net des transitions. Elle permet d'exprimer les propriétés structurelles du réseau, notamment les P-invariants et les T-invariants. Dans ce projet, le réseau de Petri sert à représenter les états critiques du cycle de tir, les ressources finies et les synchronisations entre conditions.

2.3 Logique temporelle linéaire

La logique temporelle linéaire (LTL) permet d'exprimer des propriétés sur les traces d'exécution [9]. Une trace est la suite des états traversés par le système. LTL sert donc à traduire des phrases comme “le tir et le rechargement ne se superposent jamais” ou “après une erreur, on doit pouvoir revenir au repos”.

Elle complète les invariants statiques en décrivant l'évolution attendue du système. Les opérateurs utilisés sont :

TABLE 2.1 – Opérateurs LTL utilisés

Opérateur	Symbole	Type	Signification
Globally	$G(\varphi)$	sûreté	φ est vraie dans tous les états futurs.
Finally	$F(\varphi)$	vivacité	φ sera vraie dans au moins un état futur.
Next	$X(\varphi)$	pas suivant	φ est vraie dans l'état immédiatement suivant.
Until	$\varphi U \psi$	évolution	φ reste vraie jusqu'à ce que ψ devienne vraie.

Les propriétés de sûreté affirment qu'un mauvais état n'arrive jamais ; les propriétés de vivacité affirment qu'un bon état finit par arriver. Pour un système de tir, les deux dimensions sont indispensables : il faut interdire les combinaisons dangereuses, mais aussi garantir la récupération après erreur et le retour à un état contrôlable.

2.4 Modèle d'acteurs

Le modèle d'acteurs représente la concurrence par des entités autonomes qui possèdent leur propre état et communiquent par messages asynchrones [4, 1]. Akka Typed fournit une implémentation Scala de ce modèle avec typage des messages, supervision hiérarchique et isolation des états [5]. La correspondance avec les réseaux de Petri est naturelle : un état d'acteur peut être abstrait par une place, la réception d'un message par une transition, et une synchronisation par une transition à plusieurs préconditions.

Chapitre 3

Architecture fonctionnelle du FCS

3.1 Choix d’architecture

Le FCS réunit les caractéristiques d’un système critique distribué : concurrence entre sous-systèmes, synchronisation stricte avant tir, ressources finies et tolérance aux erreurs. L’architecture Akka Typed répond à ces contraintes en isolant les responsabilités dans des acteurs typés. Chaque acteur ne traite qu’un message à la fois, ce qui évite les accès concurrents directs à un état partagé.

3.2 Acteurs

L’implémentation repose sur huit acteurs organisés autour d’un superviseur racine :

La représentation suivante est un arbre de responsabilité. Chaque ligne est un acteur Akka ; l’indentation indique que le **SupervisorActor** crée et surveille les acteurs enfants. L’objectif est d’éviter un état partagé global : chaque acteur possède son état et communique par messages.

```
SupervisorActor
|-- SensorActor
|-- TrackingActor
|-- AmmoActor
|-- CommandActor
|-- FireControlActor
|-- KafkaProducerActor
+-- KafkaConsumerActor
```

TABLE 3.1 – Responsabilités des acteurs Akka

Acteur	Responsabilité
SupervisorActor	Création, supervision et redémarrage des acteurs enfants.
SensorActor	Détection des cibles et déclenchement du cycle.
TrackingActor	Calcul balistique, verrouillage et confiance de suivi.
AmmoActor	Gestion du stock et chargement des munitions.
CommandActor	Validation des règles d’engagement.
FireControlActor	Orchestration du cycle, synchronisation des confirmations, tir, rechargement et refroidissement.
KafkaProducerActor	Publication des événements opérationnels.
KafkaConsumerActor	Collecte de l’audit trail.

3.3 Cycle nominal

Le cycle nominal part d’une détection de cible. Le `FireControlActor` déclenche ensuite trois activités : verrouillage balistique, chargement d’une munition et demande d’autorisation. Le tir n’est exécuté que lorsque les trois confirmations sont reçues :

La formule suivante condense notre choix de conception : le booléen `readyToFire` ne vaut vrai que si les trois verrous métier sont levés. Le symbole $\wedge$ signifie “et logique” ; aucune des trois conditions ne peut remplacer les deux autres.

$$readyToFire = targetLocked \wedge ammoLoaded \wedge fireAuthorized$$

Cette condition correspond directement à la transition de synchronisation *T4* du réseau de Petri.

La trace ci-dessous vulgarise le même cycle sous forme de messages. Chaque flèche représente une demande, une confirmation ou un changement de phase observé dans l’application.

```
Detection -> Tracking -> TargetLockConfirmed
      -> AmmoActor -> AmmoLoadConfirmed
      -> CommandActor -> FireAuthConfirmed
      -> readyToFire -> FireExecuted
      -> ReloadTimeout -> CooldownTimeout -> Idle
```

3.4 Gestion des erreurs et journalisation

Les principaux cas d’erreur sont le refus de verrouillage, le refus d’autorisation, l’absence de munition et l’erreur système. Chaque situation produit une réponse contrôlée : abandon du cycle ou redémarrage supervisé. Les événements sont publiés sur des topics Kafka dédiés, ce qui assure une trace exploitable des actions critiques.

TABLE 3.2 – Situations d’erreur prises en charge

Situation	Comportement
Échec de verrouillage	Le <code>TrackingActor</code> signale un échec et le cycle est abandonné.
Refus d’autorisation	Le <code>CommandActor</code> refuse le tir si les règles d’engagement ne sont pas satisfaites.
Stock épuisé	L’ <code>AmmoActor</code> signale l’impossibilité de charger une munition.
Erreur système	Le <code>SupervisorActor</code> applique une stratégie de récupération.

Chapitre 4

Modèle formel par réseau de Petri

4.1 Définition du modèle

Dans le modèle final, chaque place est nommée $P0$, $P1$, etc. pour rester compacte dans les formules. Chaque transition est nommée $T0$, $T1$, etc. pour identifier une action franchissable. Cette numérotation n'est pas arbitraire : elle sert de pont entre le diagramme, le code Scala et les résultats de vérification.

Le réseau de Petri du FCS contient 13 places et 12 transitions :

$$|P| = 13, \quad |T| = 12$$

Le symbole $|P|$ signifie “nombre de places” et $|T|$ signifie “nombre de transitions”. Le marquage initial décrit ensuite la photographie du système au démarrage, dans l'ordre des places $P0$ à $P12$:

$$M_0 = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, n, 0, 0)$$

où n représente le stock initial de munitions. Le jeton placé dans $P0$ indique que le système démarre au repos.

Le tableau des places se lit comme un dictionnaire. Les places de type contrôle décrivent une phase du cycle ; les places de type ressource comptent une quantité qui peut augmenter ou diminuer.

TABLE 4.1 – Places du réseau de Petri

Place	Nom	Type	Rôle
P0	Idle	contrôle	Système au repos.
P1	Target_Detected	contrôle	Cible détectée.
P2	Target_Locked	contrôle	Verrouillage confirmé.
P3	Ammo_Loaded	contrôle	Munition chargée.
P4	Fire_Authorized	contrôle	Autorisation accordée.
P5	Ready_To_Fire	contrôle	Préconditions réunies.
P6	Firing	contrôle	Tir en cours.
P7	Reloading	contrôle	Rechargement.
P8	Cooldown	contrôle	Refroidissement.
P9	Error_State	contrôle	Erreur détectée.
P10	Ammo_Stock	ressource	Stock de munitions.
P11	Kafka_Queue	ressource	Événements en attente.
P12	Log_Recorded	ressource	Événements journalisés.

Le tableau des transitions décrit les actions autorisées. La partie gauche de la flèche contient les places qui doivent posséder un jeton avant l'action ; la partie droite contient les places qui recevront un jeton après l'action.

TABLE 4.2 – Transitions du réseau de Petri

Transition	Nom	Préconditions et postconditions
T0	detect_target	$\{P0, P10\} \rightarrow \{P1, P10, P11\}$
T1	lock_target	$\{P1\} \rightarrow \{P2\}$
T2	load_ammo	$\{P10\} \rightarrow \{P3\}$
T3	authorize_fire	$\{P2\} \rightarrow \{P4\}$
T4	ready_sync	$\{P3, P4\} \rightarrow \{P5\}$
T5	fire	$\{P5\} \rightarrow \{P6, P11\}$
T6	end_fire	$\{P6\} \rightarrow \{P7\}$
T7	reload_complete	$\{P7\} \rightarrow \{P8\}$
T8	cooldown_complete	$\{P8\} \rightarrow \{P0\}$
T9	error_detected	$\{P0\} \rightarrow \{P9\}$
T10	error_recovery	$\{P9\} \rightarrow \{P0\}$
T11	kafka_log	$\{P11\} \rightarrow \{P12\}$

La figure donne une lecture visuelle du même modèle : les cercles sont les places, les rectangles sont les transitions, les flèches sont les préconditions et postconditions, et les jetons matérialisent l'état courant.

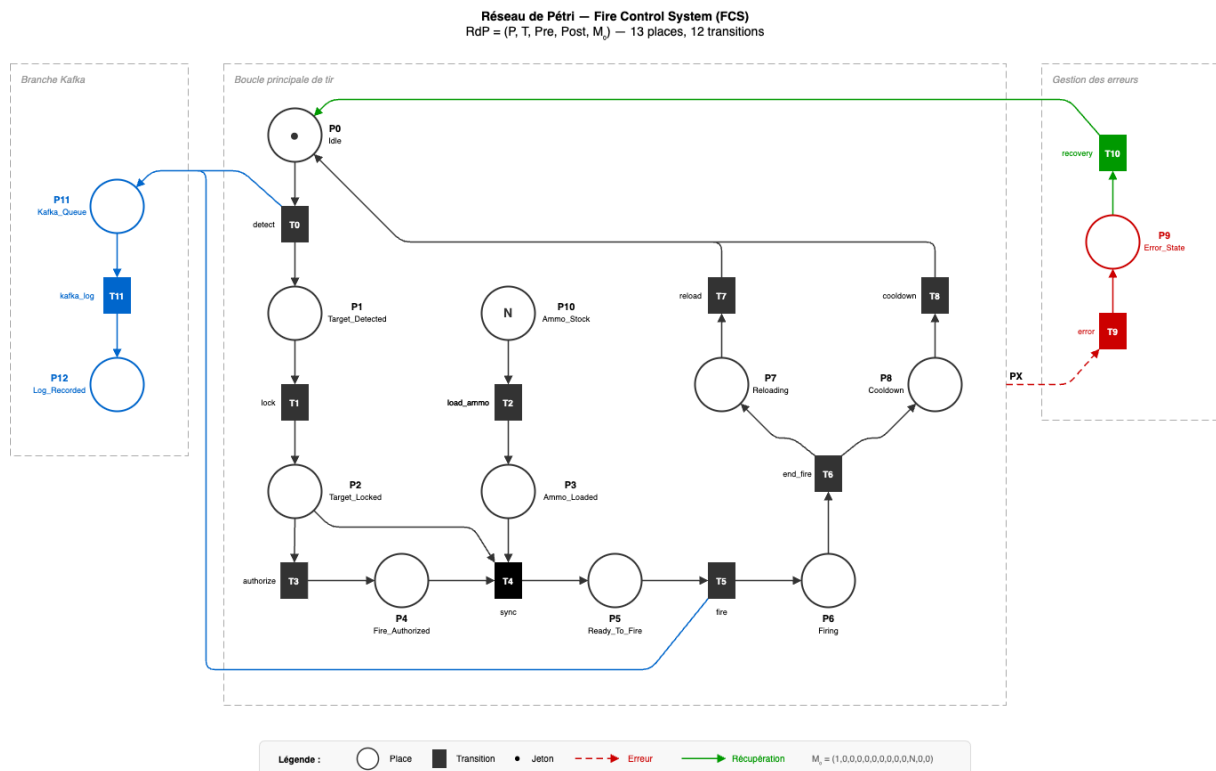


FIGURE 4.1 – Réseau de Petri du Fire Control System

4.2 Correspondance avec Akka

La traduction formelle conserve les responsabilités critiques des acteurs. Le `SensorActor` correspond à la détection, le `TrackingActor` au verrouillage, l'`AmmoActor` au chargement, le `CommandActor` à l'autorisation, le `FireControlActor` à la synchronisation et au cycle de tir, et le `SupervisorActor` au cycle d'erreur/récupération.

Le tableau suivant explicite cette traduction. Il ne cherche pas à recopier tout le code Akka ; il garde uniquement les états et actions qui ont un impact direct sur la sûreté du tir.

TABLE 4.3 – Correspondance entre acteurs et réseau de Petri

Acteur Akka	Places	Transitions
<code>SensorActor</code>	P0, P1	T0
<code>TrackingActor</code>	P1, P2	T1
<code>AmmoActor</code>	P10, P3	T2
<code>CommandActor</code>	P2, P4	T3
<code>FireControlActor</code>	P3 à P8 puis P0	T4 à T8
<code>SupervisorActor</code>	P0, P9	T9, T10
<code>KafkaProducerActor</code>	P11, P12	T11

4.3 Hypothèses d'abstraction

Le modèle formel abstrait les types précis de munitions, les calculs balistiques détaillés, les délais réels et les politiques internes de supervision. Ces choix réduisent la complexité sans supprimer les conditions de sûreté essentielles. Le modèle conserve ce qui influence directement la validité d'un tir : disponibilité d'une munition, verrouillage, autorisation, état de tir, rechargement, refroidissement, erreurs et journalisation.

Chapitre 5

Vérification formelle

5.1 Objectif de la vérification

Cette partie constitue le cœur du rapport. L’objectif est de démontrer que le modèle formel ne permet pas d’atteindre des états dangereux et que les comportements attendus restent possibles. La vérification est donc organisée autour de trois niveaux complémentaires :

- l’exploration exhaustive des marquages atteignables ;
- les invariants, qui expriment des propriétés vraies dans tous les états ;
- les propriétés LTL, qui expriment des contraintes sur l’évolution des traces.

Les tests classiques valident quelques scénarios représentatifs. La vérification formelle apporte une garantie plus forte : tous les marquages atteignables depuis M_0 sont analysés. Dans le contexte d’un système de contrôle de tir, cette exhaustivité est indispensable, car un seul état dangereux suffit à invalider le modèle.

5.2 Pipeline de vérification

Le pipeline appliqué au projet part du réseau de Petri construit dans le code Scala, puis explore son espace d’états avant de vérifier les propriétés.

Le schéma ci-dessous se lit de gauche à droite. Chaque bloc transforme le modèle en une information plus exploitable : d’abord le réseau, puis tous ses états possibles, puis les propriétés vérifiées, enfin la comparaison avec l’exécution Akka.



FIGURE 5.1 – Chaîne de vérification du FCS

La première étape produit les places, transitions, préconditions, postconditions et le marquage initial. La deuxième étape énumère les marquages atteignables par parcours en largeur. La troisième étape applique les propriétés de sûreté, de vivacité et de conservation. La dernière étape compare les traces attendues du réseau de Petri aux événements observés dans l’implémentation Akka.

5.3 Exploration de l’espace d’états

L’exploration exhaustive est effectuée par BFS avec un stock initial de trois munitions. BFS signifie *Breadth-First Search*, ou parcours en largeur : l’algorithme part de M_0 , franchit toutes les transitions possibles, puis recommence depuis chaque nouvel état découvert. Chaque nœud du graphe représente un marquage et chaque arc correspond au franchissement d’une transition.

Le tableau suivant résume donc la taille réelle du comportement étudié. Les 111 marquages sont les états effectivement atteignables, et non les états imaginés ou testés manuellement.

TABLE 5.1 – Résultats de l’exploration exhaustive

Métrique	Valeur
Marquages atteignables	111
Arcs du graphe d’accessibilité	227
Deadlocks détectés	0

Le nombre de marquages reste suffisamment faible pour permettre une analyse complète. L’absence de deadlock signifie qu’aucun marquage atteignable ne bloque totalement le système. Cette propriété est importante : un FCS ne doit pas rester dans un état sans issue, notamment après erreur, rechargement ou refroidissement.

5.4 Invariants structurels

Les P-invariants expriment des lois de conservation. On peut les lire comme des compteurs pondérés qui doivent garder la même valeur, quel que soit le chemin pris dans le réseau. Le vecteur y indique quelles places participent au compteur et avec quel poids.

Un vecteur y est un P-invariant si :

$$C^T y = 0$$

Ici C^T est la matrice d’incidence transposée : elle permet de vérifier que les gains et pertes de jetons s’annulent pour ce compteur. Dans ce cas, la quantité $y^T M$ reste constante pour tout marquage atteignable M . Ces invariants ne dépendent pas d’un scénario particulier ; ils décrivent la structure même du réseau.

Le premier invariant trouvé est le vecteur suivant. Les valeurs à 1 désignent les places prises dans le compteur ; les valeurs à 0 désignent celles qui sont ignorées :

$$y_1 = (1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0)$$

Il représente la conservation du flux de contrôle :

$$P0 + P1 + P2 + P4 + P5 + P6 + P7 + P8 + P9 = 1$$

Il garantit que le système possède exactement un état principal actif parmi les places de contrôle retenues. Cette propriété empêche les combinaisons incohérentes où le système serait simultanément au repos, en tir, en rechargement ou en erreur.

Le second invariant utilise des poids différents selon les places. Les coefficients 2 indiquent que certaines ressources ou phases comptent double dans la conservation globale :

$$y_2 = (1, 0, 0, 2, 0, 2, 1, 1, 1, 2, 1, 1)$$

Il relie les places de contrôle et les places ressource. Il montre que les consommations et productions de jetons sont cohérentes avec la conservation globale attendue du modèle. Les deux invariants sont vérifiés sur les 111 marquages atteignables.

5.5 Invariants métier

Les invariants métier traduisent les exigences de sûreté du FCS. Ils ne se limitent pas à des propriétés mathématiques abstraites : chaque invariant correspond à une interdiction ou obligation opérationnelle.

Le tableau doit être lu comme un contrat entre le modèle et le projet. La colonne informelle exprime l'idée métier ; la colonne formelle indique comment cette idée est vérifiée sur les marquages ou transitions.

TABLE 5.2 – Invariants métier vérifiés

ID	Exigence informelle	Interprétation formelle	Résultat
INV1	Pas de tir sans verrouillage	La synchronisation $T4$ dépend de $P4$, produit uniquement après verrouillage et autorisation.	OK
INV2	Pas de tir sans munition chargée	$T4$ exige $P3$, produit par $T2$ après consommation d'une munition disponible.	OK
INV3	Pas de tir sans autorisation	$T4$ exige $P4$, produit par $T3$ après validation de commande.	OK
INV4	Stock non négatif	Pour tout marquage M , $M(P10) \geq 0$.	OK
INV5	Exclusion tir/rechargement	Pour tout marquage M , $M(P6) + M(P7) \leq 1$.	OK
INV6	Pas de tir pendant refroidissement	$M(P6) > 0 \Rightarrow M(P8) = 0$.	OK
INV7	Conservation du flux de contrôle	La somme des places de contrôle principales reste égale à 1.	OK
INV8	Tout tir est journalisé	Si le tir est atteint, un événement de journalisation devient atteignable.	OK
INV9	Retour à l'état idle	Des états avec $M(P0) > 0$ restent atteignables après les cycles.	OK
INV10	Absence de deadlock	Aucun marquage atteignable n'est sans transition franchissable.	OK

Les invariants INV1, INV2 et INV3 forment le noyau de sûreté du tir : ils empêchent toute exécution de $T5$ si les préconditions opérationnelles n'ont pas été réunies. INV5 et INV6 empêchent les recouvrements dangereux entre phases incompatibles. INV4 protège la ressource finie, tandis qu'INV8 assure la traçabilité. INV10 élargit la sûreté à l'ensemble du cycle en garantissant qu'aucun état atteignable ne piège définitivement le système.

5.6 Propriétés LTL

Les invariants décrivent des états ; les propriétés LTL décrivent des traces. Cette distinction est essentielle. Un invariant peut prouver qu'un état interdit n'est jamais atteint, mais il ne suffit pas toujours à prouver qu'une action attendue finit par se produire.

Dans les formules ci-dessous, G signifie "toujours", F signifie "finalement", $\neg$ signifie "non", $\wedge$ signifie "et", et $\rightarrow$ signifie "implique". Les noms comme *firing*, *reloading* ou *idle* sont des

prédicats : ils valent vrai lorsque le marquage contient un jeton dans la place correspondante. Les formules LTL utilisées combinent donc sûreté, vivacité, récupération et traçabilité.

TABLE 5.3 – Propriétés LTL vérifiées

#	Formule	Type	Signification	Résultat
1	$G(\neg(firing\ reloading))$	$\wedge$ Sûreté	Le tir et le rechargement ne sont jamais actifs simultanément.	OK
2	$G(fire \rightarrow F(log_recorded))$	$\rightarrow$ Traçabilité	Chaque tir finit par produire une trace journalisée.	OK
3	$G(error \rightarrow F(idle))$	Récupération	Toute erreur finit par revenir vers un état idle.	OK
4	$G(F(idle))$	Vivacité	L'état idle reste toujours réatteignable.	OK
5	$G(\neg(ammo < 0))$	Sûreté	Le stock de munitions ne devient jamais négatif.	OK
6	$G(cooldown \rightarrow \neg firing)$	$\rightarrow$ Sûreté	Un état de refroidissement exclut le tir.	OK

Les propriétés de sûreté interdisent les combinaisons critiques : tir pendant rechargement, tir pendant refroidissement ou stock négatif. Elles complètent les invariants en exprimant ces contraintes sur toutes les positions d'une trace. Les propriétés de traçabilité garantissent qu'un tir ne disparaît pas silencieusement : il doit finir par produire un événement enregistré. Les propriétés de récupération et de vivacité vérifient que l'erreur n'est pas terminale et que le système conserve une capacité de retour à l'état idle.

La formule $G(error \rightarrow F(idle))$ est particulièrement importante. Elle ne dit pas seulement que l'état d'erreur est compatible avec le modèle ; elle impose que chaque occurrence d'erreur soit suivie d'un retour possible vers un état contrôlé. Pour un système supervisé, cette propriété formalise la stratégie de récupération portée par le SupervisorActor.

5.7 Bornitude

La bornitude assure qu'aucune place ne peut accumuler un nombre illimité de jetons. Vulgairement, elle répond à la question : “un compteur peut-il exploser sans limite?”. Pour un système critique, une borne finie évite les files infinies, stocks incohérents ou duplications d'état.

Le réseau est 6-borné, la borne maximale provenant de la place $P12$ qui accumule les événements journalisés.

TABLE 5.4 – Bornes maximales des places

Places	Borne	Interprétation
P0–P2, P4–P9	1	Les places de contrôle sont 1-bornées.
P3	3	Le nombre de munitions chargées est borné par le stock initial.
P10	3	Le stock ne dépasse pas sa valeur initiale.
P11	2	Deux événements peuvent être en file.
P12	6	Deux logs par cycle, sur trois cycles possibles.

Cette bornitude est acceptable pour deux raisons. D'abord, les places de contrôle restent 1-bornées, ce qui garantit l'absence de duplication incohérente de l'état principal. Ensuite, les

places ressource sont bornées par des quantités physiques ou fonctionnelles : munitions disponibles, file d'événements et logs produits.

5.8 Vivacité et T-invariant

La vivacité mesure si une transition peut encore se produire dans le futur. Le niveau L4 est le plus fort : il signifie qu'une transition reste toujours récupérable depuis n'importe quel état atteignable. Dans notre cas, il serait anormal d'exiger ce niveau pour le tir après épuisement des munitions.

Le réseau n'est donc pas vivant au sens L4 pour toutes les transitions. Après épuisement des munitions, les transitions du cycle de tir ne sont plus franchissables. Ce résultat n'est pas une erreur : il exprime une contrainte physique. Un système sans munition ne doit pas continuer à tirer.

Un T-invariant décrit une séquence de transitions dont l'effet net est nul : après avoir exécuté cette séquence, le réseau revient au même équilibre de jetons. En revanche, le cycle d'erreur reste vivant :

$$x_1 = (0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0)$$

Il correspond au T-invariant $\{T9, T10\}$: détection d'erreur puis récupération. L'effet net du cycle est nul, car le système revient de $P9$ vers $P0$. Cette propriété formalise le comportement attendu du superviseur : l'erreur est prise en compte, puis le système redevient contrôlable.

5.9 Synthèse de vérification

TABLE 5.5 – Synthèse des résultats formels

Catégorie	Résultat
Invariants métier	10/10 PASS
Propriétés LTL	6/6 PASS
P-invariants	2 trouvés et validés
T-invariants	1 trouvé et validé
Bornitude	6-borné
Vivacité	L1/L4 conforme aux attentes
Deadlocks	0

La vérification formelle confirme que le modèle satisfait les propriétés attendues. Les états dangereux liés au tir sont exclus, les ressources restent bornées, les erreurs sont récupérables et aucun deadlock n'est atteignable. Ces résultats donnent un fondement formel à l'implémentation Akka, sous réserve que celle-ci respecte les contrats modélisés.

Chapitre 6

Simulation comparée Petri Net / Akka

6.1 Méthodologie

La simulation comparée vérifie que le modèle formel et l'implémentation produisent des comportements cohérents. Elle suit trois phases : exécution déterministe des transitions du réseau de Petri, exécution des mêmes scénarios dans Akka, puis comparaison des traces via un mapping transition-événement.

Le mapping ci-dessous sert de dictionnaire entre les deux mondes. Une transition Petri représente une action abstraite ; l'événement Akka associé est ce que l'on observe dans l'application réelle pour la même décision.

TABLE 6.1 – Extrait du mapping transition / événement Akka

Transition	Événement Akka associé
T0 detect_target	SensorActor, TargetDetected, InitiateFireCycle.
T1 lock_target	TrackingActor, TargetLockConfirmed.
T2 load_ammo	AmmoActor, AmmoLoadConfirmed.
T3 authorize_fire	CommandActor, FireAuthConfirmed.
T4 ready_sync	FireControlActor, synchronisation readyToFire.
T5 fire	FireControlActor, FireExecuted.
T9/T10	SupervisorActor, erreur puis récupération.
T11 kafka_log	KafkaProducerActor, PublishEvent.

6.2 Résultats

Chaque scénario teste une décision critique du système. “Blocage” côté Petri ne signifie pas une panne : cela signifie que le modèle refuse volontairement une action dangereuse parce qu'une précondition manque.

TABLE 6.2 – Résultats des scénarios de conformité

Scénario	Petri Net	Akka	Verdict
Cycle nominal	Succès	Succès	Conforme.
Tir sans autorisation	Blocage de T4	Abandon	Conforme.
Tir sans munition	Blocage de T4	Abandon	Conforme.
Erreur et récupération	Succès	Succès	Conforme.
Épuisement des munitions	Blocage de T0	Abandon	Conforme.
Double tir	Blocage de T5	Impossible	Conforme.

Les six scénarios sont conformes. Les différences observées sont des différences d'abstraction : Akka exécute réellement des messages concurrents et des timers, tandis que le réseau de Petri représente un entrelacement non temporisé. Le point essentiel est que les décisions critiques restent identiques : synchronisation avant tir, refus des cycles incomplets, récupération après erreur et respect des ressources.

6.3 Commandes d'exécution

Le projet s'exécute avec `sbt`. Les commandes ci-dessous couvrent les trois usages du dépôt : compiler le code, vérifier automatiquement les propriétés, puis lancer les démonstrations.

TABLE 6.3 – Commandes principales

Commande	Rôle
<code>sbt compile</code>	Compilation des sources Scala.
<code>sbt test</code>	Tests unitaires et vérification formelle.
<code>sbt "run akka-demo"</code>	Démonstration du système Akka.
<code>sbt "run conformance"</code>	Vérification de conformité entre Petri net et Akka.
<code>sbt "run live"</code>	Lancement du mode interactif.

Chapitre 7

Mode live interactif

7.1 Objectif

Le mode `live` ajoute un panneau de commande interactif pour piloter le système pas à pas, observer l'état courant et distinguer les actions manuelles des actions automatiques. Il complète les modes `akka-demo` et `conformance` par une vue orientée exploitation et formation.

7.2 Fonctionnement

Le mode repose sur `Main.runInteractive` et `InteractiveSimulator`. Il construit le réseau, sélectionne le mode d'affichage, puis exécute une boucle interactive. Les transitions sont classées entre actions opérateur et actions automatiques, ce qui permet d'observer le franchissement progressif du modèle.

Le tableau suivant décrit les choix d'interface du mode live. Il ne remplace pas la vérification formelle ; il sert à rendre le modèle manipulable et compréhensible pendant une démonstration.

TABLE 7.1 – Fonctionnalités du mode live

Fonction	Description
Verbose	Journal détaillé, transitions automatiques visibles, compte à rebours explicite.
Compact	Sortie concise et invite utilisateur simplifiée.
Commandes	Numéro d'action, <code>status</code> , <code>help</code> , <code>reset</code> , <code>quit</code> .
Progression automatique	Franchissement automatique des transitions non manuelles.
Validation	Couverture par <code>InteractiveSimulatorSpec</code> .

Chapitre 8

Conclusion

Le projet met en relation une implémentation concurrente et un modèle formel vérifiable. L'architecture Akka fournit une structure réaliste pour représenter les composants distribués d'un FCS. Le réseau de Petri abstrait les comportements critiques dans un modèle fini, suffisamment précis pour exprimer les conditions de sûreté et suffisamment compact pour être exploré exhaustivement.

La contribution principale est la vérification formelle. Les dix invariants métier, les six propriétés LTL, les P-invariants, la bornitude, la vivacité attendue et l'absence de deadlock convergent vers la même conclusion : le modèle empêche les comportements dangereux retenus dans le périmètre du projet. La simulation comparée confirme ensuite que l'implémentation Akka respecte les contrats comportementaux du modèle.

Les limites tiennent aux abstractions retenues : le réseau n'est pas temporisé, le calcul balistique est simplifié, les types de munitions sont agrégés et la supervision Akka est modélisée par un cycle générique. Des extensions possibles seraient l'ajout de réseaux de Petri temporisés, une modélisation plus fine des règles d'engagement et une génération automatique de scénarios de test à partir du graphe d'accessibilité.

Annexe A

Couverture des objectifs

TABLE A.1 – Correspondance avec les objectifs du projet

Objectif	Couverture dans le rapport
État de l’art	Chapitre 1 : vérification formelle, réseaux de Petri, LTL, modèle d’acteurs.
Modélisation fonctionnelle	Chapitre 2 : architecture Akka, acteurs, flux et erreurs.
Traduction formelle	Chapitre 3 : places, transitions, hypothèses et diagramme.
Vérification de propriétés	Chapitre 4 : invariants, LTL, bornitude, vivacité, deadlocks.
Simulation et validation	Chapitre 5 : comparaison Petri Net / Akka.
Mode live interactif	Chapitre 6 : commandes, affichage et validation.

Bibliographie

- [1] Gul Agha. *Actors : A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [2] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008.
- [3] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.
- [4] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular actor formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, 1973.
- [5] Lightbend. Akka documentation. <https://doc.akka.io/>, 2024.
- [6] Jacques-Louis Lions et al. Ariane 5 flight 501 failure report, 1996. ESA/CNES.
- [7] Tadao Murata. Petri nets : Properties, analysis and applications. *Proceedings of the IEEE*, 77(4) :541–580, 1989.
- [8] James L. Peterson. *Petri Net Theory and the Modeling of Systems*. Prentice-Hall, 1981.
- [9] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science*, pages 46–57. IEEE, 1977.